

Hybrid SLC-MLC RRAM Mixed-Signal Processing-in-Memory Architecture for Transformer Acceleration via Gradient Redistribution

Chang Eun Song
University of California, San Diego
La Jolla, USA
cesong@ucsd.edu

Priyansh Bhatnagar
University of California, San Diego
La Jolla, USA
prbhatnagar@ucsd.edu

Zihan Xia
University of California, San Diego
La Jolla, USA
z5xia@ucsd.edu

Nam Sung Kim
UIUC
Champaign, USA
nam.sung.kim@gmail.com

Tajana S Rosing
University of California, San Diego
La Jolla, USA
tajana@ucsd.edu

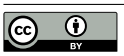
Mingu Kang
University of California, San Diego
La Jolla, USA
mingu@ucsd.edu

Abstract

Transformers, while revolutionary, face challenges due to their demanding computational cost and large data movement. To address this, we propose HyFlexPIM, a novel mixed-signal processing-in-memory (PIM) accelerator for inference that flexibly utilizes both single-level cell (SLC) and multi-level cell (MLC) RRAM technologies to trade-off accuracy and efficiency. HyFlexPIM achieves efficient dual-mode operation by utilizing digital PIM for high-precision and write-intensive operations while analog PIM for high parallel and low-precision computations. The analog PIM further distributes tasks between SLC and MLC PIM operations, where a single analog PIM module can be reconfigured to switch between two operations (SLC/MLC) with minimal overhead (<1% for area & energy). Critical weights are allocated to SLC RRAM for high accuracy, while less critical weights are assigned to MLC RRAM to maximize capacity, power, and latency efficiency. However, despite employing such a hybrid mechanism, brute-force mapping on hardware fails to deliver significant benefits due to the limited proportion of weights accelerated by the MLC and the noticeable degradation in accuracy. To maximize the potential of our hybrid hardware architecture, we propose an algorithm co-optimization technique, called *gradient redistribution*, which uses Singular Value Decomposition (SVD) to decompose and truncate matrices based on their importance, then fine-tune them to concentrate significance into a small subset of weights. By doing so, only 5-10% of the weights have dominantly large gradients, making it favorable for HyFlexPIM by minimizing the use of expensive SLC RRAM while maximizing the efficient MLC RRAM. Our evaluation shows that HyFlexPIM significantly enhances computational throughput and energy efficiency, achieving maximum 1.86× and 1.45× higher than state-of-the-art methods.

Keywords

Processing-In-Memory, LLM, AI Processor, Accelerators, Transformer, Mixed-Signal, RRAM



This work is licensed under a Creative Commons Attribution 4.0 International License.

ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731109

ACM Reference Format:
Chang Eun Song, Priyansh Bhatnagar, Zihan Xia, Nam Sung Kim, Tajana S Rosing, and Mingu Kang. 2025. Hybrid SLC-MLC RRAM Mixed-Signal Processing-in-Memory Architecture for Transformer Acceleration via Gradient Redistribution. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3695053.3731109

1 Introduction

Transformer models have brought significant advancements in a wide range of machine learning (ML) applications, including natural language processing and computer vision [61]. However, their high computational cost and memory bandwidth/capacity requirements pose considerable challenges for the hardware deployment, given the limited resources such as computing capability, memory bandwidth, energy, and delay budgets [34, 66]. These difficulties arise from the immense memory bandwidth usage and intensive data transfer cost required for model operations, making it challenging to process them efficiently with conventional accelerators such as GPUs and ASICs [45, 57, 58, 73, 77]. As an alternative solution, memory-centric processing architectures have been emerging, specifically processing-in-memory (PIM) [50, 76, 82]. These technologies eliminate data movement and leverage memory's inherent parallelism to enhance computational efficiency. In particular, both digital and analog PIM approaches have shown significant efficiency improvements - digital PIM reduces data movement costs while maintaining the accuracy for high bit-precision operations [16, 24, 32, 60, 62, 70, 81], and analog PIM demonstrates dramatic efficiency improvement, achieving more than two orders of magnitude benefit [5, 9, 17, 24] through low-voltage swing operations.

To leverage such opportunities, several efforts have explored PIM techniques that employ DRAM, SRAM, RRAM (Resistive RAM), PRAM (Phase Change Memory), and other emerging memory topologies [4, 39, 56, 72]. Out of these efforts, RRAM is one of the most widely explored devices for PIM acceleration due to its non-volatile memory (NVM) nature, high storage density, fast read operation, low energy consumption, and excellent analog programmability for the multi-level cell (MLC) [23, 39, 44, 48, 60, 62, 76, 80, 82]. Despite its great potential in terms of efficiency, practical applications of RRAM for Transformer models face multiple challenges in terms of its non-ideal characteristics and reliability concerns. While digital

RRAM PIM approaches have been actively explored for various complex models due to their reliability and precision advantages [16, 60, 62], analog RRAM PIM has been explored mostly in simpler algorithms such as multilayer perceptrons, spiking neural networks (SNNs), and convolutional neural networks (CNNs), which exhibit relatively high inherent error resiliency within the model itself [27, 33, 40, 71].

On the other hand, Transformer models consist of many cascaded layers and often require high bit precision for specific operations, making them sensitive to noise sources and resulting in immediate accuracy degradation from such noise. Due to these accuracy concerns, a large body of work [32, 70, 81] has explored digital PIM solutions for processing Transformers while reducing data movement costs. However, it has shown limited improvement due to less parallelism in General matrix-vector multiplications (GEMV) operations than analog PIMs [74], falling short of fully leveraging the substantial potential for energy and delay efficiency offered by low-swing analog operations. SPIRNT [77] explored analog PIM within a limited scope, using it as a pre-processor before the main digital processing to filter redundant tokens by calculating the correlation scores between all query and key pairs in the RRAM array. Despite a few analog efforts, the study of analog PIM acceleration for Transformers remains largely unexplored, mainly due to concerns about accuracy degradation.

One of the main advantages of analog RRAM PIM is its ability to support multi-level cell (MLC) programming, where multiple bits of information (2 to 6-bits) can be stored per cell based on programmable resistance levels, unlike single-level cell (SLC) which stores only a single bit (e.g., high vs. low). In addition, PIM with MLC achieves higher computing throughput as each memory cell processes multiple bits simultaneously. Despite above advantages, the inherent non-linearity in MLC operations leads to severe accuracy degradation. Our experiments demonstrate that even the simplest Transformer models [12] suffer catastrophic accuracy loss when implemented entirely with 2-bit MLC, e.g., 40% drop for BERT-Base with GLUE MRPC dataset.

In contrast, SLC offers a much higher noise margin against data distortion, ensuring better accuracy but at the cost of significantly greater area and energy overhead. Exploiting both types of devices (SLC & MLC) in a single hardware offers a great knob to trade-off between efficiency and accuracy by containing the critical contents in the SLC while less important and error-tolerant data in the MLC. To unlock the true potential of enhanced efficiency and large capacity from analog PIM, it is desirable to process a dominant portion of data in the MLC. However, applying these methods to Transformer encounters two major challenges: 1) the error-tolerant portion to be processed in MLC is often not sufficient enough so that the acceleration offered by the hybrid architecture does not lead to a significant improvement in efficiency, and 2) even when this is not the case, it is hard to clearly demarcate which part is error-tolerant vs. error-susceptible.

This work aims to tackle these challenges at the intersection of hardware & software by introducing a novel hybrid SLC-MLC RRAM mixed-signal architecture, termed **HyFlexPIM** along with an algorithmic transformation called *gradient redistribution*. HyFlexPIM exhibits three key characteristics (1) **mixed-signal**, leveraging both digital & analog **PIM** computations, (2) **Hybrid**, further dividing the analog PIM into SLC and MLC RRAM operations given the accuracy requirement, and (3) **Flexible**, enabling seamless switching between SLC-MLC configurations in a single PIM unit.

On the software side, our proposed gradient redistribution technique reshapes Transformer models without accuracy compromise to be more compatible with the hybrid hardware platform, rather than passively relying on the inherent error resiliency of the model itself. In this approach, we accomplish these goals by employing Singular Value Decomposition (SVD) to decompose matrices based on their order of importance. After truncating the post-SVD matrix to remove redundancy, we fine-tune it so that a small portion of the weights have dominant importance. This approach will not only facilitate a clear demarcation between critical and error-tolerant sections of the task but will also enhance the size of the error-tolerant section for efficient hardware mapping, thereby maximizing the use of low-power analog processing in the MLC while achieving higher storage density and improved computational throughput. All SVD decomposition, truncation, and fine-tuning are performed entirely in the software in advance. Meanwhile, the hardware, featuring reconfigurability for hybrid SLC-MLC operations, only utilizes the final truncated matrices for GEMV operations for inference, resulting in lower computing and storage costs.

Our detailed contributions are as follows:

- We introduce architectural and circuit-level hardware designs to support the dual-tier approach, facilitating importance-based data flow with hybrid SLC-MLC RRAM mixed-signal PIM and a reconfigurable ADC for dynamic precision.
- The gradient redistribution technique allows us to protect only the most important 5-10% & 5-20% of weights by storing them in the SLC for the encoder & decoder models, respectively, to achieve high precision while most weights are processed in the MLC.
- We performed accuracy analysis by employing realistic circuit error models to reflect the non-idealities of real RRAM devices [15], peripheral circuitry, and the detailed computational and data flow.
- We conducted extensive experiments on various benchmarks, including both encoder-based, decoder-based, and vision transformer models such as BERT [12], GPT-2 [47], Llama3 [3], and ViT-Base [14] with multiple datasets of GLUE (7 tasks) [65], WikiText-2 [37], PTB [36], and CIFAR-10 [28].

The results indicate that HyFlexPIM exhibits superior throughput and energy efficiency of maximum 1.86× and 1.24× compared to state-of-the-art (SoA) methods [31], with negligible accuracy repercussions (less than 1%) in the encoder model and vision transformer and loss increase (less than 10% of loss) in decoder model, on average.

2 Background and Motivation

2.1 Transformer

Figure 1 illustrates the computing stages of an end-to-end Transformer encoder and decoder, which consist of multiple blocks internally. Attention-based models fundamentally consist of two primary modules: Multi-Head Attention (MHA) and Feed Forward Network (FFN). The attention mechanism is designed to selectively focus on a subset of segments in the input sequence while generating an output. The core concept involves computing a weighted sum of values (**V**), where the weights are determined by the similarity

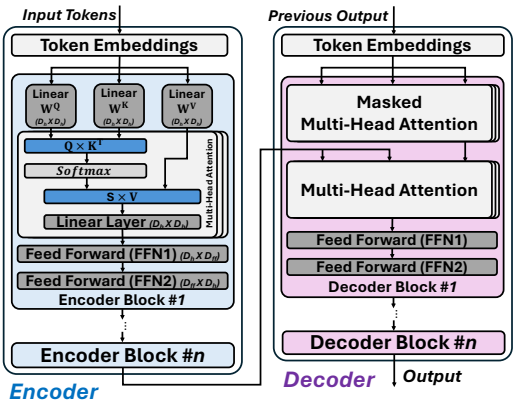


Figure 1: Variants of Transformer model architectures and dataflow: (a) an encoder and (b) a decoder architecture.

between a query (**Q**) and a set of keys (**K**). This process can be mathematically represented as follows:

$$\mathbf{Q} = \mathbf{W}^{\mathbf{Q}}(x), \quad \mathbf{K} = \mathbf{W}^{\mathbf{K}}(x), \quad \mathbf{V} = \mathbf{W}^{\mathbf{V}}(x) \tag{1}$$

where **Q**, **K**, and **V** are $L \times D_h$ dimensional matrices with L representing the sequence length, D_h as hidden dimension, and x as input embedding. Here, $\mathbf{W}^{\mathbf{Q}}$, $\mathbf{W}^{\mathbf{K}}$, and $\mathbf{W}^{\mathbf{V}}$ are learnable weight matrices for the generation of queries, keys, and values, respectively. Then, the attention scores are computed as the dot product between **Q** and **K**, followed by the application of a softmax function to obtain the normalized weights. The attention mechanism is formulated as follows with d_{head} being the size of each head:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{head}}}\right)\mathbf{V} \tag{2}$$

Typically, the attention module is followed by an additional linear layer with a weight matrix $\mathbf{W}_{\mathbf{proj}}$ for further processing. After MHA, the output is processed through a FFN consisting of two layers: FFN1 and FFN2. FFN1 performs a linear transformation using a weight matrix $\mathbf{W}_{\mathbf{ffn1}}$ of dimensions $D_h \times D_{ff}$, followed by a non-linear activation function, typically ReLU or GELU. This layer projects the D_h-dimensional input to a higher-dimensional space of size D_{ff}, resulting in an output dimension of $L \times D_{ff}$. Subsequently, FFN2 applies a linear transformation to project the intermediate representation back to the original hidden dimension size of $L \times D_h$ with a weight matrix $\mathbf{W}_{\mathbf{ffn2}}$ with dimension of $D_{ff} \times D_h$.

Figure 2 shows the distribution of the number of operations for each computation stage in transformers with respect to different sequence lengths. As a large portion (>70%) of computations in Transformers is from static weights [10, 45] and does not update them during inference, the weights written in the array can be reused over a large number of inputs without writing, which is costly due to the high writing voltage of NVM devices. For this reason, most of the Transformer's computing pipeline can be efficiently accelerated by PIM. Furthermore, this static nature enables pre-processing using singular value decomposition (SVD) (described in Section 2.3) for improved efficiency and supports the pre-storage of weights processed through SVD.

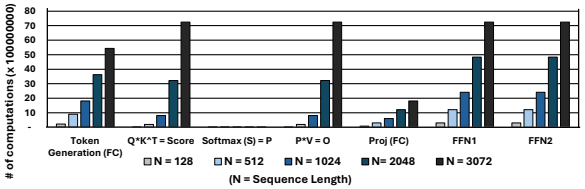


Figure 2: The number of operations for each stage in transformer models.

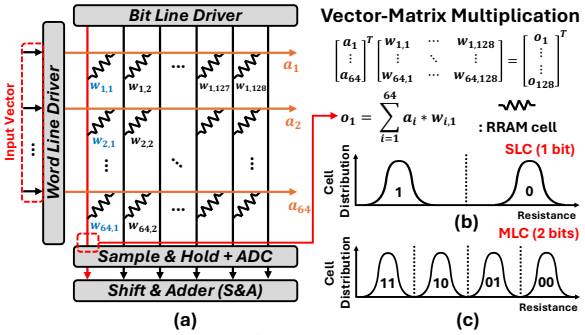


Figure 3: Illustration of (a) analog RRAM processing-in-memory vector-matrix multiplication, (b) single-level cell (SLC), (c) multi-level cell (MLC) distribution, and (d) digital RRAM processing-in-memory NOR operation.

2.2 Analog RRAM Processing In-Memory

Analog processing-in-memory (PIM) has been extensively adopted for accelerating a wide range of applications with high demands for memory and computational resources, exploiting its significant parallelism, low-power computation with low-voltage swing, and capability to minimize explicit data movement. While various memory topologies, including RRAM [11, 20, 31, 32, 52, 77, 79], PCM [69], SRAM [2], and DRAM [4], have been explored, this paper focuses on RRAM due to its suitability for both storage and computation. RRAM's non-volatile memory (NVM) nature, high storage density, low read power, and excellent analog programmability make it a highly suitable choice. Consequently, RRAM has been one of the most widely employed devices for the acceleration of highly parallel analog General matrix-vector multiplications (GEMV).

During GEMV PIM operations, each element of the matrix is mapped to the conductance (=1/resistance) of the memristor in a bitcell. While single-level cells (SLC) contain one bit per cell (Figure 3(b)), MLCs store multiple bits (e.g., 2 bits or more) per cell as multi-level conductance (Figure 3(c)). Each element of input vector is applied to the all wordlines (WLs) of the RRAM array simultaneously, horizontal lines in Figure 3(a), by the WL driver as a voltage signal. When each row of the array receives one bit of the corresponding input element in a bit-sequential manner, and assuming SLC (each storing 1 bit), each bitcell performs 1-bit by 1-bit binary multiplication. Conversely, if the cells are configured as MLCs, each bitcell performs 1-bit by multi-bit multiplications achieving higher throughput. The multiplication results from every cell are directly accumulated on the bitline (vertical line) of the array in the analog domain, generating the output of the GEMV. The following equation formally represents the GEMV operation with an input vector $\mathbf{A}_{1\times n}$ and matrix $\mathbf{W}_{n\times m}$ on an RRAM crossbar

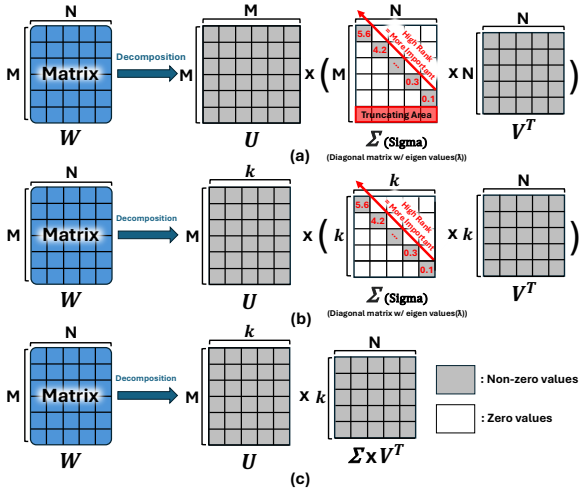


Figure 4: Post-SVD matrix applied to static weights in the transformer (a) during training, (b) after truncation, and (c) during inference.

array:

$$o_j = \sum_{i=1}^{n} a_i \cdot w_{ij} \tag{3}$$

where w_{ij} is a conductance corresponding to the element at the i-th row and j-th column of the weight matrix $\mathbf{W}$ whereas o_j is an j-th elements of output vector when an input vector ($\{a_1, ..., a_n\}$) was applied. Finally, the accumulated analog current (o_j) is captured by a sample-and-hold circuit, and subsequently converted to digital form through an analog-to-digital converter (ADC). The final result of the multi-bit GEMV operation is obtained via weighted-sum of all output bits using a shift and add (S&A) module to represent different bit positions of input. (Figure 3 (a))

Despite its great potential for an efficient GEMV operation via full parallelism from all bitcells, practical applications of RRAM for Transformer models face multiple challenges due to its non-ideal characteristics and reliability concerns. These challenges include spatial mismatch from process variations, non-linear and asymmetric weight updates, limited endurance, and restricted data retention capabilities. Intuitively, SLC offers a higher noise margin against various noise sources [15] leading to better accuracy, but at the cost of significantly greater area, delay, and energy overhead compared to MLC.

2.3 Singular Value Decomposition (SVD)

SVD is a widely utilized method [1, 35] for decomposing weight matrices into three smaller matrices while retaining much of the original information. Any matrix $W \in \mathbb{R}^{M \times N}$ can be transformed as follows:

$$W = U \Sigma V^T \tag{4}$$

As described in Figure 4 (a), U and V^T are $M \times M$ and $N \times N$ orthogonal matrices, respectively, and Σ is an $M \times N$ diagonal matrix with non-negative real numbers on the diagonal. This decomposition facilitates the analysis and manipulation of the matrix W based on the diagonal elements, known as singular values, of Σ, which indicate the importance of the corresponding dimensions, e.g., the rows of U and the columns of V^T. To reduce dimensionality, as shown in Figure 4 (b), a truncated SVD is commonly used: $W \approx W_k = U_k \Sigma_k V_k^T$, where U_k, Σ_k, and V_k contain only the first k columns, singular values, and rows, respectively. After truncation, $\Sigma_k V_k^T$ is pre-computed and stored as a matrix of size $(k \times N)$, reducing both data volume and computation cost, as illustrated in Figure 4 (c). While these SVD processes are typically performed in software, the hardware focuses on storing and utilizing the truncated matrices for inference. Although SVD is widely employed for data compression through truncation, proposed algorithm repurpose it to reshape the matrices, maximizing the efficiency of the proposed hardware, as detailed in Section 4.

3 HyFlexPIM

In this section, we introduce the detailed architecture of HyFlexPIM, which is a mixed-signal RRAM PIM accelerator designed to efficiently support both SLC and MLC PIM operations with negligible overhead. This hybrid hardware creates a great synergy with our proposed *gradient redistribution* technique to be detailed in the following Section 4, enabling effective mapping of critical weights to SLC and less important weights to MLC.

3.1 HyFlexPIM Hardware Architecture

Digital PIM, which has been extensively studied [22, 31, 74], achieves data movement reduction with reliable computation through simple bit-wise operations (e.g., NOR, INV, and others) but suffers from limited parallelism compared to analog PIM. While analog RRAM PIM offer superior parallelism, their non-ideality and expensive MLC write operation make them less suitable for storing real-time generated vectors with high bit-precision. Therefore, we opt to process attention computations in digital SLC RRAM arrays, which serve as both efficient storage and computing units. On the other hand, the static weights in the linear layers are processed in the analog PIM for the enhanced efficiency by utilizing MLC.

As described in Figure 5, HyFlexPIM includes 24 Processing Units (PUs) to enable high parallelism in processing 24 layers and has a single input and output (IO) to receive input sequences and uses a global bus to communicate with each PU inside the HyFlexPIM. Each PU is dedicated to the computation of a single layer of the encoder or decoder, allowing the PUs to work in a pipelined fashion to process multiple streamed inputs. Once a specific layer is computed, the result is transferred to the next PU for the next layer's processing. Note that the output of each layer is merely the size of $(D_h \times 1)$, maintaining the inter-PU movement cost negligible. While mapping each PU to a single layer provides optimal performance, our architecture allows flexible model scaling in several ways as follows. 1) For longer sequences (e.g., >8192) or higher hidden dimensions, multiple PUs collaboratively process a single layer to provide sufficient storage capacity, following tensor parallelism. 2) For models with fewer layers (e.g., <24), multiple PUs work collectively to compute a single layer in parallel, as seen in [31, 32, 52], again following tensor parallelism. 3) When the number of layers exceeds the available PUs, multiple chips are cascaded to support deeper layers, following pipeline parallelism. In cases 1) and 2), partial sums (<3 KB per PU) are transferred between PUs and aggregated in the Special Function Unit (SFU) via global

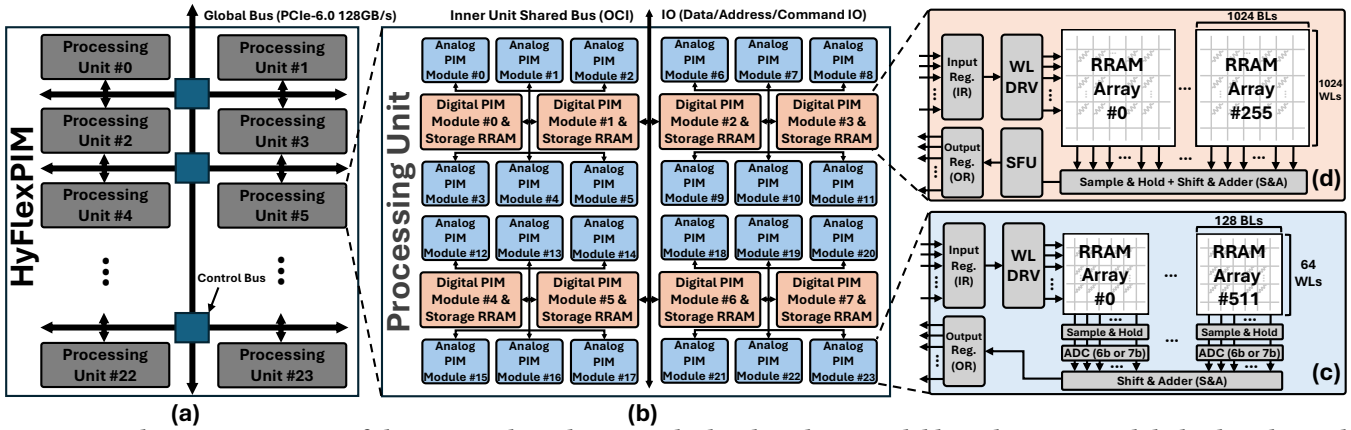


Figure 5: Architecture overview of the proposed HyFlexPIM, which is based on a scalable architecture with hybrid analog and digital RRAM PIM modules.

bus (128 GB/s [55] as will be shown in Section 5.4) with negligible latency overhead (24 cycles). For case 3), only a single hidden layer output (0.75 - 2 KB at hidden dimension of 768-2048) is transmitted between chips to continue processing the next layer, requiring less than 6-16 cycles over the PCIe-6.0 interconnect. The throughput analysis of these cases are given in Figure 17 with real benchmarks.

Figure 5 (b) shows the detailed structure of the PU, which comprises eight digital RRAM PIM modules for attention and non-linear operations and 24 analog RRAM PIM modules to accelerate the other linear operation stages. Each module is interconnected via *inner unit shared bus* to transfer the intermediate results from each stage to the next as input. An analog PIM module (Figure 5 (c)) consists of 512 RRAM arrays, each of which includes 64×128 bitcells and peripheral circuitry such as Input/Output Registers (IR/OR), Sample and Hold (S&H), 6-b/7-b flexible ADC, Shift and Adder (S&A), Word Line Drivers (WL DRV) for bitwise input fetching, and conventional read circuitry and write driver (not shown for simplicity). Digital PIM module (Figure 5 (d)) is mostly similar to the analog PIM, but does not require the analog peripheral circuitry such as ADC. IR holds the control signals of bit line and WL selectors as [31, 81]. The Digital PIM module also incorporates a Special Function Unit (SFU) to efficiently handle non-linear operations such as Softmax, Layer Normalization, and GELU. To enable these operations, we implemented max search, subtraction, exponentiation (via Taylor series), addition, division, multiplication, and square root functions, all fully pipelined using floating-point arithmetic. After computation, the results are converted to the integer format for further processing with negligible overhead as shown in Table 2. We configured each SFU to process 256 inputs per cycle to balance the throughput of GEMV operations. The digital PIM module contains 256 RRAM arrays, each of which includes 1024 bitcell per row. In the $\mathbf{Q} \times \mathbf{K}$ computation, which involves INT8 × INT8 multiplication, each output generation requires 64 NOR operations. Each NOR operation takes three columns for two input operand bits and one output bit [22]. Each row processing takes total five cycles (four cycles for writing and one cycle for reading). Based on these constraints, 256 inputs per cycle for SFU hit the optimal balance with GEMV throughput, e.g., 256 × 1024 / (64 × 3) / 5 = 273 operations. The area and energy of SFU are in Table 2. Note,

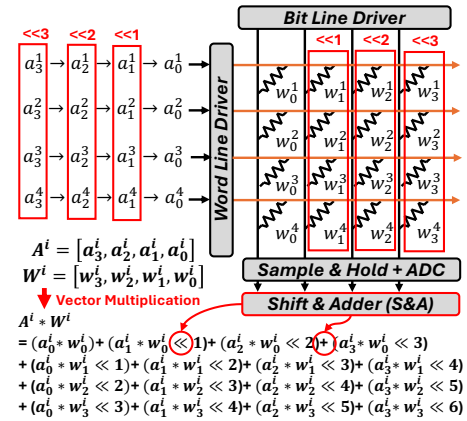


Figure 6: SLC PIM mapping and Shift & Adder operation.

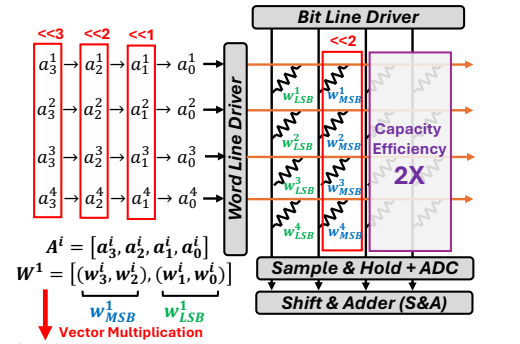


Figure 7: MLC PIM mapping and Shift & Adder operation.

our approach is optimized for the inference, with proposed SVD fine-tuning (described in Section 4) conducted prior to deployment on the hardware as a one-time software process, ensuring no additional hardware overhead for SVD processing or memory usage in inference.

3.2 MLC vs. SLC Operations

The static weights are pre-loaded into the analog RRAM PIM modules before the inference begins, based on their type of storage format (e.g., SLC vs. MLC), using conventional write drivers and a wordline (WL) driver, which are omitted in Figure 5 (c) for simplicity. Note that the varying levels of resistance for MLC are written by iteratively applying pulses through the WL based on the desired resistance level [49]. As a result, the same wordline drivers are used for both SLC and MLC, incurring no additional hardware costs to support both formats. Figure 6 and Figure 7 show the detailed view of Figure 5(c) for SLC and MLC configurations to describe the PIM dot-product operation with 4-bit weights for exposition purpose. The weights in a column of the matrix are stored vertically in the RRAM array. The SLC stores bits for the weight across adjacent columns, e.g., 4-bit words are stored across four columns. Each bit of the input is given sequentially through the WL. Depending on the bit position of each column and the cycle of input, corresponding binary weights are applied in the digital shift & adder (S&A) module. The MLC operation is mostly similar, but each bitcell contains multi-bits, e.g., 2-bit in Figure 7. Therefore, the corresponding impact factor of 1×, 4×, 16×, ... are given for the consecutive columns in the S&A module. Note that the support of SLC and MLC does not require two distinct hardware, but sharing the same hardware in a reconfigurable manner with only minor additional functions in the digital S&A module and reconfigurable ADC.

By Kirchhoff's current law, the currents accumulated by bitlines represent the dot-product results in the analog domain. As described in Figure 8 (a), analog signals from 128 bitlines are first captured by sample and hold (S&H) circuits, then sequentially selected through a multiplexer before being fed to a shared successive-approximation-register (SAR) ADC. The SAR ADC performs analog-to-digital conversion using binary search algorithm by enabling capacitors in sequence ($C_7 \rightarrow C_1$) for iterative voltage comparisons, where each comparison determines one bit of the digital output. The required ADC precision for MVM operations is given by $\lceil \log_2(R) \rceil + w - 1$, where R is the number of rows in the crossbar, w is the bits stored in a bitcell. In the SLC case, an 6-bit ADC is needed to achieve the full bit precision assuming 64 rows. In the MLC PIM operation, as shown in Figure 7, two consecutive bits of weights are packed and mapped to a single 2-bit RRAM device. In this way, MLC improves efficiency with doubled throughput, increased storage density, and halved analog computation energy. Since we still send input bits one by one, the ADC resolution required for MLC only increases by one (6-b → 7-b) compared to that of the SLC operation. While numerous studies have explored truncating bit precision of ADC to reduce area and energy consumption [31, 32], Transformers are highly sensitive to such a bit truncation, leading to significant accuracy degradation. To mitigate accuracy degradation, we employ a full-precision ADC, configured as 6-b for SLC and 7-b for MLC, in a flexible bit-precision manner. As shown in Figure 8 (b), our ADC is designed to support up to 7-b resolution. However, by simply bypassing the comparison for the largest capacitor (C_7), which corresponds to the MSB, the ADC can efficiently operate as a 6-b ADC without consuming additional power. Importantly, the energy and area overhead of this design compared to a dedicated 6-b ADC takes negligible (<1%) portion in the overall architecture. Generally, increasing the ADC precision by one bit results in doubling

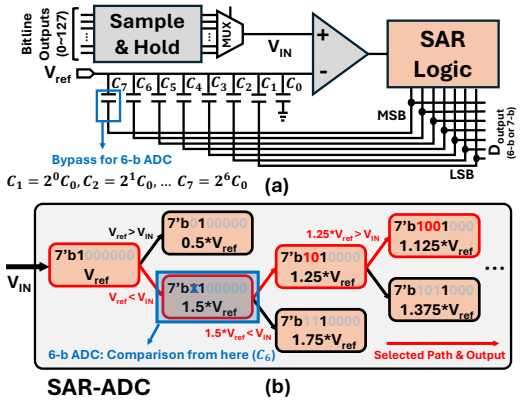


Figure 8: (a) 6 and 7 bits flexible SAR-ADC schematic, and (b) operation of SAR-ADC with reconfiguration.

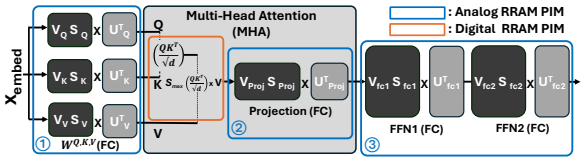


Figure 9: The proposed mapping of transformer operations to the hybrid digital-analog RRAM PIM architecture.

the energy consumption [29, 42]. However, because the number of results generated to be converted by ADC is reduced by half in the case of 2-b MLC, the total energy consumption for the ADC does not increase as compared to the SLC case. Based on reliability considerations, we adopt 2-b MLC to balance efficiency and reliability since measurements from a real RRAM chip [15, 63] show that higher-level MLCs (e.g., 3-b/4-b MLC) have a 7× higher bit error rate than SLC, causing severe accuracy degradation in Transformer models [12, 47].

3.3 Transformer Algorithm to HW Mapping

Figure 9 illustrates the dataflow of Transformer including the four fully-connected (FC) layers, and MHA layer and their mapping on PIM. After applying SVD, truncation on the linear matrices ($\mathbf{W^Q}$, $\mathbf{W^K}$, and $\mathbf{W^V}$), and fine-tuning, the matrices are divided into two matrices: $V \times \Sigma$ and U^T (Since input is multiplied with transposed version of $\mathbf{W}$) and stored in the analog PIM RRAM array for the inference as shown in Figure 9 block ①. After matrix multiplication with an input (X_{embed}) with these two matrices, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ tokens are generated. The generated tokens or outputs are stored in the digital PIM module via the inner-unit shared bus, where partial sums from different sub-arrays are accumulated to finalize the GEMV computation before passing the results to the next stage. While recent analog RRAM PIM works tend to employ the SLC due to the non-ideality concerns [31, 32], our approach utilizes more efficient MLCs for the most (e.g., 90 - 95% for BERT) of less critical elements by relying on the proposed gradient redistribution with minimal impact (<1%) on accuracy as detailed in Section 6.1. The same process is applied on FC layer ② in the MHA and two FFN layers ③ by storing their static weights in the RRAM array and performing the PIM operation by applying the output of previous layer as an input.

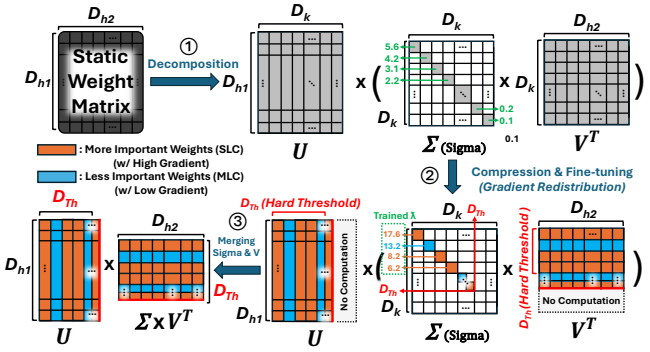


Figure 10: Hard-Thresholding and Fine-tuning the post-SVD matrix.

While the weights of the above layers are static, the computations of $\mathbf{Q} \cdot \mathbf{K}^{\mathbf{T}}$ and $\times \mathbf{V}$ (in the orange box) are not static, i.e., both operands are computed in real-time given input. Therefore, we do not apply gradient redistribution to avoid a costly SVD computation on the fly, in contrast to static weights where SVD and gradient redistribution are pre-computed offline. We also employ only SLC to avoid the costly MLC writing operation, which requires iterative verify-read and write operations to reach the exact target conductance. Furthermore, this stage has relatively lower computation volume and requires high precision computations, e.g., $\times \mathbf{V}$ stage requires an operand to be 12 bits, to guarantee accuracy for a wide range of applications. For this reason, we employ digital PIM for this section (orange box) as proposed in [22, 25, 31, 75] to avoid any potential accuracy degradation and excessive ADC costs. Although the benefit of digital PIM is limited compared to analog PIM, it still bypasses the expensive data movement cost for the KV cache from the memory to the processor. We also store the intermediate computed results in the digital PIM with an SLC format.

As described in Figure 1, there are two distinct types of Transformer tasks: the encoder and decoder stages. In the encoder (or the prefill phase of the decoder), multiple input vectors are applied to PIM through the wordlines over multiple cycles to perform matrix-matrix multiplications. In contrast, during the decoder phase, a single input vector is applied for vector-matrix multiplication, generating one token at a time. Nevertheless, the PIM operations remain the same, ensuring consistency in execution.

4 SVD-based Gradient Redistribution

This section introduces the SVD-based *gradient redistribution* technique, designed to maximize the efficiency of the HyFlexPIM accelerator. By applying SVD and truncation, we expose the importance of weight elements and concentrate critical weights into a smaller subset through fine-tuning. This hardware-algorithm co-design preserves critical weights in SLC while utilizing efficient MLC RRAM for the majority of less-critical weights, maximizing the HyFlexPIM architecture efficiency.

4.1 Post-SVD Truncation

As mentioned in Section 2.3, by applying SVD, the importance of weight data becomes more discrete due to the ordered singular values in Σ. This facilitates mapping critical data to SLC RRAM, enhancing robustness and accuracy, while less critical data can be

Algorithm 1 Hybrid SLC-MLC RRAM Analog PIM Simulation

Require: Weight matrix $W \in \mathbb{R}^{m \times n}$, desired rank k, and optimizer (AdamW)

1: **Step 1: SVD Decomposition**
2: Perform SVD on W: $W = U\Sigma V^\top$ where $U \in \mathbb{R}^{m \times m}$, $\Sigma \in \mathbb{R}^{m \times n}$, $V \in \mathbb{R}^{n \times n}$
3: **Step 2: Truncated SVD**
4: Calculate target rank $k = \frac{m \times n}{m+n}$
5: Truncate matrices: $U \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $V \in \mathbb{R}^{k \times n}$
6: **Step 3: Fine-tuning**
7: **while** training **for** 1-3 epochs **do**
8: Perform optimizer step (e.g., AdamW)
9: Store gradients and update weights
10: **end while**
11: **Step 4: Gradient-based Rank Selection**
12: Identify top singular values and corresponding dimensions with the highest gradient magnitudes
13: **Step 5: Mapping Weights into PIM array**
14: Map identified weights to SLC, other weights to MLC

mapped to MLC RRAM, optimizing area, latency, and energy efficiency. However, direct application of SVD increases computational overhead due to the additional matrix multiplications. To address this issue, we employ a hard-threshold for Σ that maintains the same computational complexity as the pre-SVD matrices. Figure 10, step ②, illustrates the computation reduction process. For a weight matrix W of size $D_{h1} \times D_{h2}$, the total number of MAC computations before SVD is $L \times D_{h2} \times D_{h1}$, and the total number of parameters is $D_{h1} \times D_{h2}$. We truncate the Σ to keep the limited number of ranks. Then, we combine Σ and V^T, e.g., $\Sigma \times V^T$, to reduce the number of parameters for the inference, as described in Figure 10, step ③. Consequently, the number of MAC operations changes to $L \times D_{h2} \times D_{Th} + L \times D_{Th} \times D_{h1}$, where D_{Th} is the lower rank. To ensure that the computational load remains the same as the original model, we replace D_{Th} with a hard threshold $\frac{D_{h1} \times D_{h2}}{D_{h1}+D_{h2}}$. By using the hard threshold ($\frac{D_{h1} \times D_{h2}}{D_{h1}+D_{h2}}$), the total number of parameters for $\Sigma \times V^T$ and U is also preserved before and after SVD.

The low-rank truncation causes some accuracy degradation. For example, in BERT-Large, the MRPC task shows a 16% accuracy drop, RTE shows a 20% drop, and in GPT-2, the loss increases by more than 2×. To recover accuracy after truncation, we fine-tune the model by re-training for 1-3 epochs, as described in **Steps 1-3** of Algorithm 1, which we found sufficient to maintain accuracy and loss comparable to the pre-SVD case for transformer models used in our evaluation. Simpler tasks often required only a single epoch of fine-tuning, effectively balancing computational efficiency with parameter reduction. Moreover, the SVD decomposition, truncation, and fine-tuning (**Steps 1 - 4** in Algorithm 1) are performed entirely in software, while the final matrices $V \times \Sigma$ and U^T are offloaded to the hardware for GEMV operations. Thus, the fine-tuning process incurs only one-time cost without adding any hardware overhead.

4.2 Gradient Redistribution

While the fine-tuning in **Step 3** aims to recover the accuracy, it also performs a crucial role, *gradient redistribution*, which helps to increase the portion of error-tolerant parts dramatically, facilitating

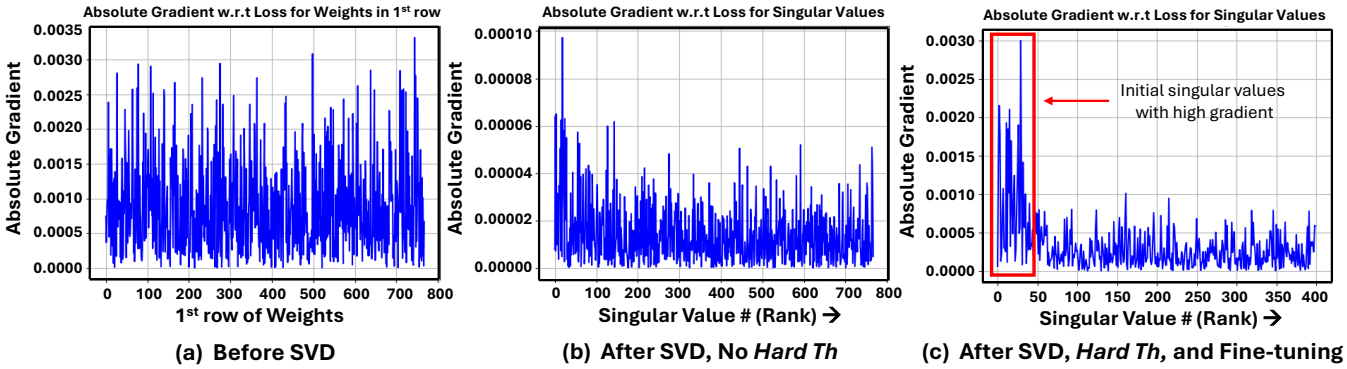


Figure 11: Gradient distribution change for FC layer in BERT-Base: (a) gradient of each weight (order-wise) in a row before SVD, (b) gradient after SVD, without applying hard threshold, (c) gradient after SVD, applying hard threshold and gradient redistribution.

the use of MLC for most of computation. The magnitude of the gradient of the loss function with respect to a specific weight in a neural network provides vital insights into the importance of the weight. In addition, the gradient of the loss function $\mathcal{L}$ with respect to a weight w_i, denoted $\frac{\partial \mathcal{L}}{\partial w_i}$, signifies how sensitive the loss is to changes in the weight. A large gradient implies that a small change in the weight will result in a significant change in the loss, thus highlighting the weight's critical importance to the model's performance. During training, weights are updated in the direction that minimizes the loss function.

Figure 11 shows the gradient redistribution resulting from the fine-tuning process. Since there is no notion of rank before applying SVD, we examined the gradients of the weights in a row in Figure 11 (a). The gradients across all weights are rather uniformly distributed, making it challenging to identify which ranks are more sensitive to loss. After applying SVD (Figure 11 (b), which is not hard-thresholded yet), the differences in gradients among the ranks remain insufficiently distinct to clearly indicate their importance. However, by fine-tuning the weights as shown in Algorithm 1 **Step 3**, in the post-SVD matrix, the gradients are re-distributed towards the higher ranks, resulting in only a small portion of ranks having significantly higher magnitude. As illustrated in Figure 11 (c), the initial singular values now exhibit much higher gradients compared to the lower ranks, clearly indicating which ranks should be assigned to SLC for precise computation (Algorithm 1 **Step 4**). We attribute this redistribution to the fine-tuning process that attempts to recover the loss of information from the truncated ranks by putting more information on the untruncated ranks. In this process, the ranks with higher singular values tend to gain more information than the others, as these ranks are principal key components to represent the matrix.

Note above nature helps not only the clear demarcation between error-resilient (on MLC) and error-tolerant parts (on SLC), but also facilitate more computations in MLC, dramatically improving the energy, throughput, and area efficiencies. By storing the gradient distribution after fine-tuning the truncated decomposition, we analyze the gradients for the singular values of the Σ matrix. We choose singular values with the top k% (k = 0, 5, 10, 30, 40, 50, 100) gradient magnitudes to perform the computation on SLC, while the rest is computed using MLC (Algorithm 1 **Step 5**). In this manner, we use a hybrid approach to minimize the accuracy degradation and maximize efficiency. We study the selection of optimal k further in Section 6.

5 Methodology

5.1 Software Benchmarks

Table 1: Hyper-parameters for fine-tuning.

Model	Batch Size	Learning Rate	Optimizer
BERT-Base	32	2e-5	AdamW
BERT-Large	32	5e-6	
GPT-2	2	2e-5	
Llama3	2	2e-5	
ViT-Base	10	5e-6	

The proposed HyFlexPIM is evaluated on various Transformer-based models for both NLP and CV domains including the encoder models BERT-Base and BERT-Large, the decoder model GPT-2-Small (GPT-2) and Llama-3.2-1B (Llama3), and the vision transformer model (ViT-Base) for object detection. For the encoder models, 7 datasets from the General Language Understanding Evaluation (GLUE) benchmark [64], which includes cola, mrpc, qnli, qqp, rte, sst-2, and sts-b, are used for the evaluation. The maximum sequence length (MSL) for all GLUE datasets is set to 128. The decoder model GPT-2 is tested on the WikiText-2 dataset [38] with an MSL of 1024, and Llama3 is tested on PTB [36] with an MSL of 100. ViT-Base is evaluated on the CIFAR-10 dataset with an input resolution of 224×224 and a patch size of 16×16. Throughout this work, we employ INT8 for all the linear layers and for **Q**, **K**, and **V** in the attention mechanism, while FP16 is used for other non-linear operations which are computed by HyFlexPIM's SFU. Table 1 shows the batch size, learning rate, and optimizer used for fine-tuning Transformer models. All the fine-tuning tasks were conducted on a server with a 12-core Intel Xeon Silver 4214R CPU and two NVIDIA GeForce RTX A6000 GPUs.

5.2 RRAM PIM Non-ideality Considerations

We developed a simulator to evaluate the non-idealities of analog behavior in SLC-MLC PIM modules along with other accuracy

impacts from SVD truncation, fine-tuning, and the distribution of analog and digital PIM modules. The simulator also models the serial bit computing mechanism and shift-and-add operations to accurately capture the accuracy impact of this specific operation order. This simulation incorporates the noisy RRAM behavior with non-ideal resistance deviating from the target. Specifically, we injected noise into the parameters of all linear layers, following the formula:

$$\tilde{W} = W \odot (1 + \eta), \tag{5}$$

where η is a Gaussian noise tensor with the same size as the weight tensor W, $\odot$ is element-wise multiplication, $\tilde{W}$ is the noisy weight tensor used for inference. The noise level was derived from the studies by Wan et al. [63] and Fan et al. [15], utilizing the non-idealities measured from the fabricated real MLC RRAM chips. Using the experimental data from 3 million RRAM cells in [15], which showed a bit error rate (BER) of approximately 4.04% after one day of programming, we reverse-calculated the standard deviation of the Gaussian noise η in (5) to match the BER of real analog RRAM devices. On top of the RRAM device noise, we also considered the limited ADC resolution for the outputs.

Analog RRAM module processes static weights, enabling multiple inferences after a single write operation and thus being free from endurance concerns. For the digital PIM module, where real-time generated $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$, and intermediate data are supposed to be written, could potentially compromise RRAM endurance [74]. However, given large amount of RRAM capacity in HyFlexPIM and typical RRAM endurance of 10^8 cycles [18], HyFlexPIM ensures sustainable operation beyond typical server lifespans (3-5 years) even with 10K daily inference requests.

5.3 Hardware Baseline

We benchmark our HyFlexPIM against 1) the latest study in analog-digital hybrid PIM architecture for Transformer models ASADI [31] using FP32, which only uses SLC PIM to contrast our effort to use the MLC for most computations; 2) ASADI†, using INT8 precision for linear layers which provides conservative comparison by improving ASADI's efficiency; 3) SPRINT [77], which utilizes the digital processor with RRAM storage for the linear layer computation while using analog PIM for the limited part of attention (MSB 4-bit computation of $\mathbf{Q} \cdot \mathbf{K}$); 4) a near-memory processing (NMP) baseline, TransPIM [81], which is based on the function-in-memory DRAM (FIMDRAM) processor [30] with HBM; and 5) a non-PIM baseline that processes data in a digital module consisting of INT8 dot product units derived from SPRINT [77], after moving data from 6.28 GB off-chip DRAM to on-chip SRAM cache, assuming large enough cache capacity. For direct comparison, all results from these baselines are scaled to the same 65nm process technology, following the methodology in [59]. For energy estimation, we utilize NVSIM [13] for RRAM, ARM Memory Compiler with high-density 65 nm single-port SRAM (version r0p0) [6] with 1 GHz operating frequency, and DRAM access energy from [68]. The near-memory processing bank energy is from [81] while HBM-related data is sourced from [43]. We evaluate ASADI and SPRINT's methodologies within our hardware to assess their effectiveness in our optimized mixed-signal PIM architecture.

Table 2: The hardware configuration and component-level area, power of HyFlexPIM.

Component	Area (mm^2)	Power (mW)	Parameter	Value
Analog RRAM Module				
RRAM Array	0.048 (9.6%)	60.78 (6.5%)	Bit/Cell Size Total	1-b/2-b 64×128 512
IR	0.00065 (0.1%)	0.13 (0.01%)	Size/each	64B
OR	0.00129 (0.3%)	0.53 (0.1%)	Size/each	128B
WL DRV	0.02 (3.7%)	297.71 (32%)	Resolution Total	1-b 64×512
ADC	0.30 (64.2%)	512.00 (55%)	Resolution Total	6-b/7-b 512
S&A	0.10 (22%)	59.54 (6.4%)	Total	512
S&H	6e-5 (0.01%)	12e-6 (0.001%)	Total	512
Sum	0.47	930.69	-	-
Total	**11.24**	**22,336.59**	-	24
Digital RRAM Module				
RRAM Array	2.86 (35.8%)	3,890.02 (59.6%)	Bit/Cell Size Total	1-b 1024×1024 256
IR	0.0031 (0.04%)	0.76 (0.01%)	Size/each	1KB
OR	0.0032 (0.04%)	1.65 (0.03%)	Size/each	1KB
WL DRV	0.14 (1.8%)	2,381.64 (36.5%)	Resolution Total	1-b 1024×256
S&A	0.21 (2.6%)	119.08 (1.8%)	Total	1024
S&H	13e-5 (0.002%)	23e-6 (0.0%)	Total	1024
SFU	4.79 (59.8%)	138.89 (2.1%)	# inputs	256
Sum	8.01	6,532.05	-	-
Total	**64.05**	**52,256.41**	-	8

5.4 HyFlexPIM Configuration

HyFlexPIM is designed to store all original and intermediate data within RRAM arrays. This also makes HyFlexPIM's memory capacity be dependent on the input sequence length. The HyFlexPIM is configured to handle the MSL of 8192. For BERT-Large has 24 layers, with each PU assigned to process a single layer. On the other hand, BERT-Base and GPT-2, having 12 layers, benefits from a 2× throughput increase with our accelerator. The analog PIM module comprises 512 arrays of 64×128 (1KB) RRAM for INT8 $\mathbf{W^Q}$, $\mathbf{W^K}$, and $\mathbf{W^V}$, as well as Proj (FC), FFN1, and FFN2 matrices. Meanwhile, the digital PIM module includes 256 arrays of 1024×1024 (128KB) RRAM for intermediate matrices.

We adopt the data transfer and on-chip communication topologies in [51], which also used full-flow RRAM. Inter-PU data transfer is facilitated by a 1000 GB/s On-Chip Interconnect (OCI) [26] and a PCIe-6.0 interface with 128 GB/s bandwidth for outer-PU communication [55]. Detailed specifications of the RRAM arrays are provided in Table 2. The analog modules utilize 1-bit (SLC) or 2-bit (MLC) one transistor and one memristor (1T1M) RRAM cells, while the digital modules use only 1-bit 1T1M RRAM arrays. Area and power configurations for the 1T1M RRAM arrays are derived from [31, 63]. RRAM arrays read/write data in a column-parallel manner, with SET/RESET voltages of 1.62/3.63 V for 1-bit cells [21]. Specifically, the RRAM features an on-state resistance (R_{ON}) of 6 kΩ with an on/off ratio of 150 [79]. To manage 6-bit/7-bit data across 64 rows using 1-bit/2-bit cells, a shared 6-bit/7-bit reconfigurable ADC is employed for 128 columns. The bitline currents are captured in 128

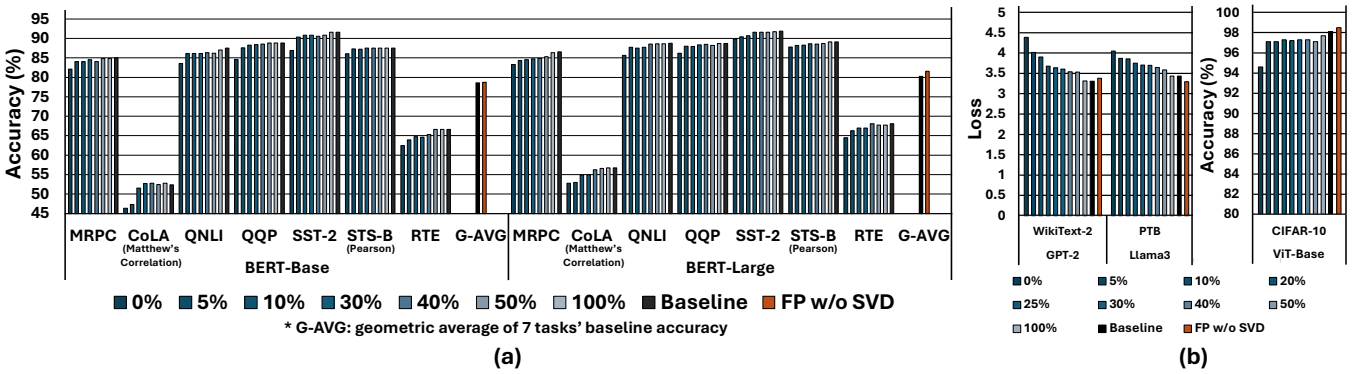


Figure 12: Accuracy trends with respect to the SLC rate for (a) BERT-Base, BERT-Large, (b) GPT-2, Llama3, and ViT.

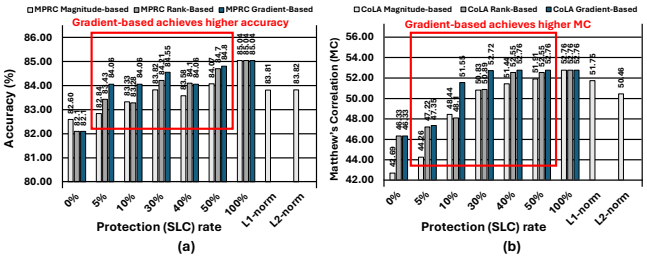


Figure 13: Accuracy comparison between weight magnitude-based vs. rank-based after SVD vs. gradient-based after SVD SLC selections in BERT-Base with (a) MRPC and (b) CoLA datasets.

sample-and-hold circuits [29, 53]. During the following 100 ns cycle, these analog values are sequentially input into a single 1.28 GSps ADC unit, while the crossbar initiates its next read operation in a pipelined manner. Consequently, 128 bitline currents are converted to digital within 100 ns before latching the subsequent set.

The area and power specifications for the S&A unit and on-chip SRAM buffers (IR and OR) are sourced from [31, 54], with the DRV details obtained from a 1-bit digital-analog converter (DAC) in [31, 51]. We validate the behavior of RRAM arrays using a modified NVSIM [13]. Additionally, we developed an error-noise injected simulator to evaluate the accuracy of HyFlexPIM across various models and datasets as described in Section 5.2. All values have been scaled to a 65 nm technology node based on [59]. We use a 65 nm general-purpose standard cell library to synthesize SFU at 1 GHz operating frequency with Cadence Genus 19.1.

6 Evaluation

6.1 Accuracy Recovery by Mapping on SLC

Figure 12 illustrates the accuracy of each transformer models resulting from the hybrid SLC-MLC RRAM mapping technique with different protection rates. k% protection rate implies that weights corresponding to the singular values with top k% of the absolute gradients will be mapped on SLC, while the rest will be mapped on MLC. The baseline represents the accuracy with INT8 precision without in-memory computing noise, but with SVD decomposition and fine-tuning. FP w/o SVD refers to models that use floating-point without applying SVD decomposition. The accuracies of FP w/o SVD for BERT are from [12], GPT-2 from [46], and ViT from [78], while we fine-tuned Llama3. For the BERT-Base (Figure 12 (a)), it is observed that maintaining a 5% SLC rate in MRPC, QNLI, and STS-B tasks minimizes accuracy degradation to less than 1% compared to the baseline model with quantization. In CoLA, QQP, SST-2, and RTE by increasing the SLC rate to 10-30%, the accuracy drop is maintained to be less than 1% of the baseline. In the case of BERT-Large, the QNLI, QQP, STS-B, and RTE tasks require only a 5% SLC rate while the others maintain the accuracy with 10-30% rate, confirming greater robustness against errors from MLC compared to smaller models. In Figure 12 (b), we assessed the evaluation loss on the WikiText-2 and PTB dataset for GPT-2 and Llama3 models, respectively. With an SLC rate of 20%, we observe less than a 10% increase in loss compared to the case with a 100% SLC rate. Furthermore, for the ViT-Base on the CIFAR-10 dataset, we observe that an SLC rate of only 5% shows less than 1% accuracy drop. These findings indicate that strategic selection of SLC rates is crucial for maintaining high accuracy across different tasks and model sizes.

6.2 Gradient vs. Rank vs. Magnitude based SLC mapping

We further investigated the advantages of gradient-based rank selection on SVD over 1) weight magnitude-based selection without SVD, and 2) brute-force selection of the initial top singular values after SVD (rank-based). Figure 13 shows that for MRPC and CoLA tasks in the BERT-Base model, the gradient-based approach consistently outperforms both methods. This is because the other two methods lack a direct connection to the loss while the gradient-based approach leverages sensitivity to the training loss, ensuring better selection of critical parameters, ultimately leading to higher accuracy and robustness.

6.3 Comparsion with Modern Accelerators

We evaluate HyFlexPIM against ASADI [31], ASADI†, SPRINT [77], NMP baseline [81], and a non-PIM baseline using GLUE and WikiText-2 datasets on BERT-Large and GPT-2 models. All comparison baselines feature distinct memory hierarchies, compute logic, on-/off-chip memory storage capacity, and in-memory computing capabilities with different benchmarks. For the fair comparison with ASADI, we maintained the same digital and analog PIM memory capacity as HyFlexPIM; however, ASADI utilizes only SLC, unlike

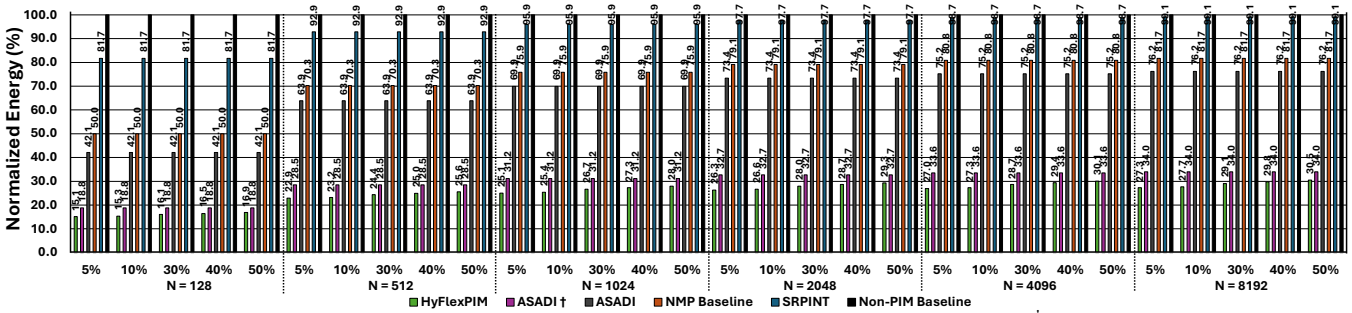


Figure 14: Normalized energy consumption of linear layers compared to the energy of ASADI†, ASADI [31], NMP baseline, SPRINT [77], and non-PIM baseline.

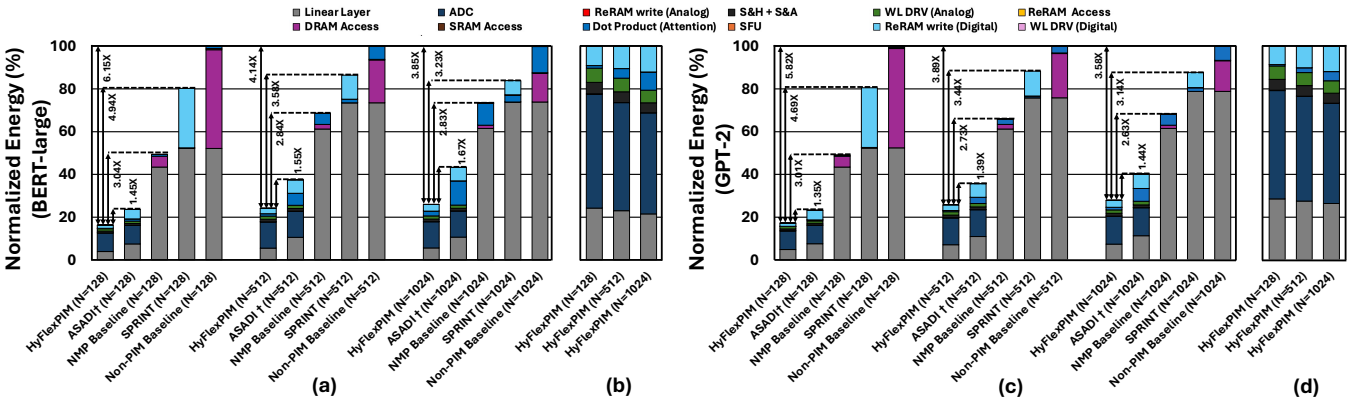


Figure 15: End-to-end energy: (a) comparison with SOA, and (b) breakdown with HyFlexPIM at 5% SLC for BERT-Large, and (c) comparison with SOA, and (d) breakdown with HyFlexPIM at 30% SLC for GPT-2.

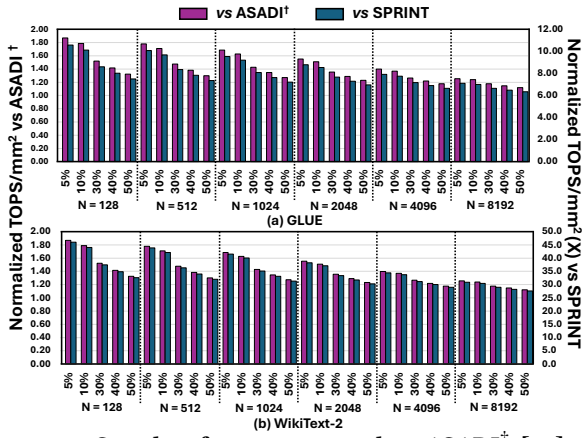


Figure 16: Speedup factor compared to ASADI† [31] and SPRINT [77] with respect to the SLC rate and the sequence length.

HyFlexPIM, while benefiting from token pruning. The token pruning ratio from [31] is applied to reflect its throughput and energy benefits. SPRINT [77] reports absolute energy and delay values for each operation including the digital processor, memory access, and in-memory token pruning ratio. These values are utilized to estimate the energy and delay overheads. Non-PIM baseline also exploits the energy and delay numbers of digital processor reported in [77], while SRAM and DRAM energy and delay numbers are from simulators described in Section 5.3. The NMP baseline [81] reports absolute energy values for near-bank processing and HBM access, which are exploited for energy estimation. Figure 14 shows linear layer energy efficiency, and Figure 15 (a, c) presents end-to-end energy comparison, while Figure 15 (b, d) provides HyFlexPIM's energy breakdown across different sequence lengths. Additionally, Figure 16 compares throughput with ASADI† and SPRINT.

6.3.1 ***HyFlexPIM VS ASADI.*** ASADI introduces RRAM PIM hardware to accelerate end-to-end Transformer operations with FP32, employing a compression method designed to exploit data locality in attention computations. Their focus is primarily on accelerating the attention mechanism, while computing linear layers using analog RRAM PIM exclusively with SLC, without exploiting MLC's superior efficiency. On the other hand, HyFlexPIM specifically targets the acceleration of linear layers, which represent a substantial computational bottleneck. For linear layer operations (Figure 14), our approach achieves maximum 1.24× higher energy efficiency compared to ASADI† (modified version to use INT8 for linear layers for fair comparison) with 5% SLC rate, despite ASADI†'s potential accuracy degradation from INT8 quantization. These benefits are more pronounced with shorter sequence lengths where FFN layer becomes the main bottleneck. Considering end-to-end performance, HyFlexPIM achieves 1.1 - 1.86× speedup compared to ASADI† (Figure 16), primarily due to our 2-bit MLC implementation doubling the throughput with the same energy consumption. At N=1024,

we demonstrate maximum 1.67× and 1.44× end-to-end energy efficiency improvements for BERT-Large and GPT-2, respectively (Figure 15 (a,c)), mainly due to ASADI's FP32 precision overhead in attention computations. Furthermore, our approach offers nearly 1.9× higher area efficiency compared to ASADI†. With approximately 20% SLC usage, the accuracy drop is less than 1% in most cases, while achieving an average of 1.23× improvement in linear layer energy efficiency and 1.79× speedup. Notably, HyFlexPIM achieves greater benefits with moderate sequence lengths, aligning well with recent trends in maintaining moderate effective sequence lengths through techniques like RAG models, state-space-based approaches, and selective attention [7, 8, 19, 41, 67].

6.3.2 ***HyFlexPIM VS SPRINT.*** We compare HyFlexPIM with the SPRINT, which performs in-memory token pruning using RRAM PIM and executes remaining computations using a traditional digital processor. For linear layer operations, as shown in Figure 14, HyFlexPIM achieves maximum 5.4× energy reduction compared to SPRINT. In terms of end-to-end performance, HyFlexPIM demonstrates maximum 4.94× and 4.69× energy efficiency improvements for GLUE and WikiText-2 respectively at N=128 (Figure 15), while achieving 10.6× and 46× speedup (Figure 16), even when applying SPRINT's 74.6% sparsity to only SPRINT's attention operations. On average with 20% SLC rate, HyFlexPIM achieves a 5.34× reduction in linear layer energy consumption, while providing 10.1× and 44× end-to-end speedup for GLUE and WikiText-2 datasets, respectively. The benefits arise from the highly parallel HyFlexPIM architecture and minimized data movements. Note that SPRINT can only reduce the data movement of attention computation. Therefore, the proposed HyFlexPIM obtains higher speedup especially at the lower length of sequences, where the data movement and computation of FFNs dominate.

6.3.3 ***HyFlexPIM VS NMP Baseline.*** While NMP reduces data movement cost compared to conventional DRAM-to-cache transfers, it still relies on auxiliary computing units located near the memory, rather than performing computation directly within the memory arrays. As a result, NMP suffers from higher energy and delay overheads from the data movement and digital computation compared to HyFlexPIM. As shown in Figure 14, HyFlexPIM achieves up to 3.31× higher energy efficiency than the NMP baseline [30] for linear operations. The end-to-end energy efficiency comparison in Figure 15, demonstrates 4.94× and 4.69× improvements for GLUE and WikiText-2, respectively.

6.3.4 ***HyFlexPIM VS Non-PIM Baseline.*** For linear layer operations, HyFlexPIM achieves a maximum 6.6× energy efficiency improvement over the non-PIM baseline (Figure 14). The end-to-end energy efficiency comparison shows 6.15× and 5.82× improvements for GLUE and WikiText-2 respectively (Figure 15 (a, c)). These substantial gains are attributed to two main factors in the baseline: high energy costs from frequent DRAM off-chip data movements and inefficient GEMV operations in conventional digital processors without the benefits of PIM.

6.3.5 ***HyFlexPIM scalability.*** As described in Section 3.1, HyFlexPIM provides flexibility and scalability through 1) tensor parallelism, where multiple PUs collaborate to enhance throughput, and 2) pipeline parallelism across multiple chips. For example, tensor

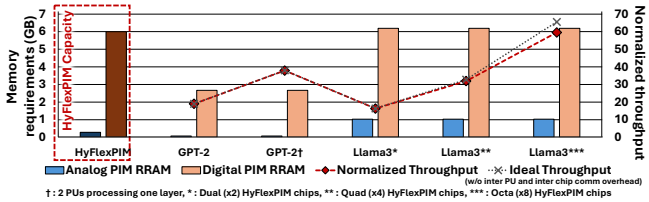


Figure 17: Memory requirements and throughput scalability of HyFlexPIM for N = 8192.

parallelism is applied to smaller models such as GPT-2, where two PUs collaboratively process a single layer, improving throughput by 1.99× compared to the case without parallelism, as shown in Figure 17. This results in only a minor degradation compared to the ideal 2× scalability, attributed to the PU-to-PU partial sum transfer overhead. Conversely, for large-scale models such as Llama3, a single PU cannot hold each layer due to its large hidden dimension so that two PUs process a layer. It also requires pipeline parallelism as a chip cannot accommodate the entire model, necessitating two chips at minimum. We analyze Llama3 dual/quad/octa-chip configurations. Quad and octa chips achieve 1.96× and 3.65× throughput improvements compared to the dual-core case including the chip-to-chip communication overhead to deliver the hidden layer to the next chip. This results in only a minor degradation compared to the ideal 2× and 4× scaling benefits.

7 Related Works

Over the last decade, RRAM PIM has been explored to the neural network acceleration [11, 20, 31, 32, 52, 77, 79]. The pioneering works, ISAAC [52] and [20] demonstrate a pipelined RRAM crossbar and hybrid RRAM capable of efficiently executing CNNs. PRIME [11] introduces a reconfigurable RRAM PIM architecture, where the same PIM hardware can be configured as both regular memory and computing elements. More recently, RRAM PIM has been applied to accelerate Transformer models. For instance, SPRINT employs analog RRAM PIM to compute correlation scores as a pre-processor for pruning unimportant tokens in the attention mechanism [77]. However, the primary computation is still carried out by energy-intensive digital processors. Moreover, SPRINT only accelerates the attention mechanism, leaving other computation- and memory-intensive tasks, such as FFNs, unoptimized. Similarly, HARDSEA [32] utilizes analog RRAM PIM solely to predict token relevance. ASADI [31] introduces a hybrid analog-digital RRAM PIM hardware and a compression method designed to exploit data locality in attention computing. However, ASADI relies exclusively on SLC RRAM, thereby missing the potential benefits of a high-density and energy-efficient MLC RRAM PIM architecture. Importantly, most existing RRAM PIM approaches depend on the inherent error resilience of neural networks to circumvent the challenges posed by noisy analog RRAM PIM operations. In contrast, our approach seeks to proactively reshape the model to enhance its robustness, with principles that can be broadly applied to other memory topologies.

8 Conclusion

Analog processing-in-memory (PIM) offers significant potential for energy and speed improvements in Transformer acceleration through parallel processing and multi-level cell (MLC) capabilities, while digital PIM provides reliable computation for dynamic

weights updated in real-time. Despite these advantages, deploying PIM effectively for sophisticated models such as Transformers remains challenging due to various practical considerations. To maximize the benefits of both domains, we propose a mixed-signal approach that strategically assigns operations based on their characteristics, utilizing digital PIM for dynamic weights and analog PIM with both SLC and MLC for static weight processing.

Rather than relying passively on inherent error resiliency, our gradient redistribution technique proactively reshapes models to be more compatible with the hybrid hardware platform. This approach enables us to maximize MLC utilization over SLC in analog PIM while preserving critical computations in SLC, achieving efficient processing while maintaining model accuracy. We expect this comprehensive approach will enable practical deployment of mixed-signal PIM technology in future large-scale AI systems.

Acknowledgments

This work was supported in part by PRISM and CoCoSys, centers in JUMP 2.0, an SRC program sponsored by DARPA #434690, and by National Research Foundation of Korea (NRF) grants funded by the Korean government (MSIT) (RS-2024-00347090 and RS-2024-00405857).

References

[1] Hervé Abdi. 2007. Singular value decomposition (SVD) and generalized singular value decomposition. *Encyclopedia of measurement and statistics* 907, 912 (2007), 44.

[2] Amogh Agrawal, Akhilesh Jaiswal, Deboleena Roy, Bing Han, Gopalakrishnan Srinivasan, Aayush Ankit, and Kaushik Roy. 2019. Xcel-RAM: Accelerating binary neural networks in high-throughput SRAM compute arrays. *IEEE Transactions on Circuits and Systems I: Regular Papers* 66, 8 (2019), 3064–3076.

[3] Meta AI. 2024. Llama 3.2: Multilingual Large Language Models. https://www.llama.com.

[4] Mustafa F Ali, Akhilesh Jaiswal, and Kaushik Roy. 2019. In-memory low-cost bit-serial addition using commodity DRAM technology. *IEEE Transactions on Circuits and Systems I: Regular Papers* 67, 1 (2019), 155–165.

[5] Tanner Andrulis, Joel S Emer, and Vivienne Sze. 2023. RAELLA: Reforming the arithmetic for efficient, low-resolution, and low-loss analog PIM: No retraining required!. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–16.

[6] ARM. 2021. Artisan Memory Compilers. https://developer.arm.com/ip-products/physical-ip/embedded-memory. Accessed: 2021-11-08.

[7] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).

[8] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*. PMLR, 2206–2240.

[9] Weidong Cao, Yilong Zhao, Adith Boloor, Yinhe Han, Xuan Zhang, and Li Jiang. 2021. Neural-pim: Efficient processing-in-memory with neural approximation of peripherals. *IEEE Trans. Comput.* 71, 9 (2021), 2142–2155.

[10] Zhiyang Chen, Yousong Zhu, Zhaowen Li, Fan Yang, Chaoyang Zhao, Jinqiao Wang, and Ming Tang. 2024. The Devil is in Details: Delving Into Lite FFN Design for Vision Transformers. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 4130–4134. doi:10.1109/ICASSP48485.2024.10447756

[11] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: a novel processing-in-memory architecture for neural network computation in ReRAM-based main memory. In *Proceedings of the 43rd International Symposium on Computer Architecture* (Seoul, Republic of Korea) *(ISCA '16)*. IEEE Press, 27–39. doi:10.1109/ISCA.2016.13

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[13] Xiangyu Dong, Cong Xu, Yuan Xie, and Norman P. Jouppi. 2012. NVSim: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 31, 7 (2012), 994–1007. doi:10.1109/TCAD.2012.2185930

[14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*.

[15] Keming Fan, Wei-Chen Chen, Sumukh Pinge, H. S. Philip Wong, and Tajana Rosing. 2024. Efficient Open Modification Spectral Library Searching in High-Dimensional Space with Multi-Level-Cell Memory. arXiv:2405.02756 [cs.AR] https://arxiv.org/abs/2405.02756

[16] Fei Gao, Georgios Tziantzioulis, and David Wentzlaff. 2019. Computedram: In-memory compute using off-the-shelf drams. In *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 100–113.

[17] Sujan K Gonugondla, Mingu Kang, and Naresh R Shanbhag. 2018. A variation-tolerant in-memory machine learning classifier via on-chip training. *JSSC* 53, 11 (Nov. 2018), 3163–3173.

[18] Alessandro Grossi, Elisa Vianello, Mohamed M. Sabry, Marios Barlas, Laurent Grenouillet, Jean Coignus, Edith Beigne, Tony Wu, Binh Q. Le, Mary K. Wootters, Cristian Zambelli, Etienne Nowak, and Subhasish Mitra. 2019. Resistive RAM Endurance: Array-Level Characterization and Correction Techniques Targeting Deep Learning Applications. *IEEE Transactions on Electron Devices* 66, 3 (2019), 1281–1288. doi:10.1109/TED.2019.2894387

[19] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).

[20] Wei-Hsing Huang, Tai-Hao Wen, Je-Min Hung, Win-San Khwa, Yun-Chen Lo, Chuan-Jia Jhang, Huna-Hsi Hsu, Yu-Hsiana Chin, Yu-Chiao Chen, Chuna-Chuan Lo, et al. 2023. A nonvolatile AI-edge processor with 4MB SLC-MLC hybrid-mode ReRAM compute-in-memory macro and 51.4-251TOPS/W. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 15–17.

[21] Je-Min Hung, Cheng-Xin Xue, Hui-Yao Kao, Yen-Hsiang Huang, Fu-Chun Chang, Sheng-Po Huang, Ta-Wei Liu, Chuan-Jia Jhang, Chin-I Su, Win-San Khwa, et al. 2021. A four-megabit compute-in-memory macro with eight-bit precision based on CMOS and resistive random-access memory for AI edge devices. *Nature Electronics* 4, 12 (2021), 921–930.

[22] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. Floatpim: In-memory acceleration of deep neural network training with high precision. In *Proceedings of the 46th International Symposium on Computer Architecture*. 802–815.

[23] Yuning Jiang, Peng Huang, Zheng Zhou, and Jinfeng Kang. 2019. Circuit design of RRAM-based neuromorphic hardware systems for classification and modified Hebbian learning. *Science China Information Sciences* 62 (2019), 1–19.

[24] Hai Jin, Cong Liu, Haikun Liu, Ruikun Luo, Jiahong Xu, Fubing Mao, and Xiaofei Liao. 2021. ReHy: A ReRAM-based digital/analog hybrid PIM architecture for accelerating CNN training. *IEEE Transactions on Parallel and Distributed Systems* 33, 11 (2021), 2872–2884.

[25] Hai Jin, Cong Liu, Haikun Liu, Ruikun Luo, Jiahong Xu, Fubing Mao, and Xiaofei Liao. 2022. ReHy: A ReRAM-Based Digital/Analog Hybrid PIM Architecture for Accelerating CNN Training. *IEEE Transactions on Parallel and Distributed Systems* 33, 11 (2022), 2872–2884. doi:10.1109/TPDS.2021.3138087

[26] Norman P Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, et al. 2021. Ten lessons from three generations shaped google's tpuv4i: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1–14.

[27] Dong Eun Kim, Aayush Ankit, Cheng Wang, and Kaushik Roy. 2023. SAMBA: sparsity aware in-memory computing based machine learning accelerator. *IEEE Trans. Comput.* 72, 9 (2023), 2615–2627.

[28] Alex Krizhevsky. 2009. *Learning multiple layers of features from tiny images*. Technical Report.

[29] Lukas Kull, Thomas Toifl, Martin Schmatz, Pier Andrea Francese, Christian Menolfi, Matthias Braendli, Marcel Kossel, Thomas Morf, Toke Meyer Andersen, and Yusuf Leblebici. 2013. A 3.1 mW 8b 1.2 GS/s single-channel asynchronous SAR ADC with alternate comparators for enhanced speed in 32 nm digital SOI CMOS. *IEEE Journal of Solid-State Circuits* 48, 12 (2013), 3049–3058.

[30] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, et al. 2021. 25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2 tflops programmable computing unit using bank-level parallelism, for machine learning applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. IEEE, 350–352.

[31] Huize Li, Zhaoying Li, Zhenyu Bai, and Tulika Mitra. 2024. ASADI: Accelerating Sparse Attention Using Diagonal-based In-Situ Computing. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 774–787.

[32] Shiwei Liu, Chen Mu, Hao Jiang, Yunzhengmao Wang, Jinshan Zhang, Feng Lin, Keji Zhou, Qi Liu, and Chixiao Chen. 2023. Hardsea: Hybrid analog-reram clustering and digital-sram in-memory computing accelerator for dynamic sparse self-attention in transformer. *IEEE Transactions on Very Large Scale Integration*

(VLSI) Systems (2023).

[33] Shuang Liu, JJ Wang, JT Zhou, SG Hu, Qi Yu, TP Chen, and Yang Liu. 2023. An area- and energy-efficient spiking neural network with spike-time-dependent plasticity realized with SRAM processing-in-memory macro and on-chip unsupervised learning. *IEEE Transactions on Biomedical Circuits and Systems* 17, 1 (2023), 92–104.

[34] Siyuan Lu, Meiqi Wang, Shuang Liang, Jun Lin, and Zhongfeng Wang. 2020. Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer. In *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*. IEEE, 84–89.

[35] Xiuqing Lv, Peng Zhang, Sunzhu Li, Guobing Gan, and Yueheng Sun. 2023. Lightformer: Light-weight transformer using svd-based weight transfer and parameter sharing. In *Findings of the Association for Computational Linguistics: ACL 2023*. 10323–10335.

[36] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building a Large Annotated Corpus of English: The Penn Treebank. *Computational Linguistics* 19, 2 (1993), 313–330. https://aclanthology.org/J93-2004/

[37] Stephen Merity. 2021. The WikiText Long Term Dependency Language Modeling Dataset. https://blog.salesforceairesearch.com/the-wikitext-long-term-dependency-language-modeling-dataset/. Accessed: 2021-11-08.

[38] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843 [cs.CL]

[39] Naoko Misawa, Tao Wang, Chihiro Matsui, and Ken Takeuchi. 2024. Embedded Transformer Hetero-CiM: SRAM CiM for 4b Read/Write-MAC Self-attention and MLC ReRAM CiM for 6b Read-MAC Linear & FC Layers. In *2024 IEEE International Memory Workshop (IMW)*. 1–4. doi:10.1109/IMW59701.2024.10536980

[40] R Mohan et al. 2023. Processing-in-Memory (PIM) Based Defect Prediction of Metal Surfaces Using Spiking Neural Network. 中械工程刊 44, 4 (2023), 379–388.

[41] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. 2024. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143* (2024).

[42] Boris Murmann. [n. d.]. ADC Performance Survey 1997-2024. [Online]. Available: https://github.com/bmurmann/ADC-survey.

[43] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W Keckler, and William J Dally. 2017. Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 41–54.

[44] Mirko Prezioso, Farnood Merrikh-Bayat, Brian D Hoskins, Gina C Adam, Konstantin K Likharev, and Dmitri B Strukov. 2015. Training and operation of an integrated neuromorphic network based on metal-oxide memristors. *Nature* 521, 7550 (2015), 61–64.

[45] Yubin Qin, Yang Wang, Dazheng Deng, Zhiren Zhao, Xiaolong Yang, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. 2023. FACT: FFN-Attention Co-optimized Transformer Architecture with Eager Correlation Prediction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 22, 14 pages. doi:10.1145/3579371.3589057

[46] A. Radford, Jeffrey Wu, R. Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners.

[47] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[48] Misbah Ramadan, Nicolás Wainstein, Ran Ginosar, and Shahar Kvatinsky. 2019. Adaptive programming in multi-level cell ReRAM. *Microelectronics Journal* 90 (2019), 169–180.

[49] Misbah Ramadan, Nicolás Wainstein, Ran Ginosar, and Shahar Kvatinsky. 2019. Adaptive programming in multi-level cell ReRAM. *Microelectronics Journal* 90 (2019), 169–180. doi:10.1016/j.mejo.2019.06.004

[50] Yiming Ren, Bobo Tian, Mengge Yan, Guangdi Feng, Bin Gao, Fangyu Yue, Hui Peng, Xiaodong Tang, Qiuxiang Zhu, Junhao Chu, et al. 2023. Associative learning of a three-terminal memristor network for digits recognition. *Science China Information Sciences* 66, 2 (2023), 122403.

[51] Mehdi Saberi, Reza Lotfi, Khalil Mafinezhad, and Wouter A Serdijn. 2011. Analysis of power consumption and linearity in capacitive digital-to-analog converters used in successive approximation ADCs. *IEEE Transactions on Circuits and Systems I: Regular Papers* 58, 8 (2011), 1736–1748.

[52] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: a convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *Proceedings of the 43rd International Symposium on Computer Architecture* (Seoul, Republic of Korea) *(ISCA '16)*. IEEE Press, 14–26. doi:10.1109/ISCA.2016.12

[53] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: a convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *SIGARCH Comput. Archit. News* 44, 3 (jun 2016), 14–26. doi:10.1145/3007787.3001139

[54] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 14–26.

[55] Debendra Das Sharma. 2020. PCI Express® 6.0 Specification at 64.0 GT/s with PAM-4 signaling: a low latency, high bandwidth, high reliability and cost-effective interconnect. In *2020 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 1–8.

[56] Clinton W Smullen, Vidyabhushan Mohan, Anurag Nigam, Sudhanva Gurumurthi, and Mircea R Stan. 2011. Relaxing non-volatility for fast and energy-efficient STT-RAM caches. In *2011 IEEE 17th International Symposium on High Performance Computer Architecture*. IEEE, 50–61.

[57] Chang Eun Song, Yidong Li, Amardeep Ramnani, Pulkit Agrawal, Purvi Agrawal, Sung-Joon Jang, Sang-Seol Lee, Tajana Rosing, and Mingu Kang. 2024. 52.5 TOPS/W 1.7 GHz Reconfigurable XGBoost Inference Accelerator Based on Modular-Unit-Tree with Dynamic Data and Compute Gating. In *2024 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 1–2.

[58] Chang Eun Song, Ashkan Moradifirouzabadi, Tajana Rosing, and Mingu Kang. 2024. Efficient Transformer Acceleration via Reconfiguration for Encoder and Decoder Models and Sparsity-Aware Algorithm Mapping. In *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design* (Newport Beach, CA, USA) *(ISLPED '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3665314.3670798

[59] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.

[60] Nishil Talati, Saransh Gupta, Pravin Mane, and Shahar Kvatinsky. 2016. Logic design within memristive memories using memristor-aided loGIC (MAGIC). *IEEE Transactions on Nanotechnology* 15, 4 (2016), 635–650.

[61] M Onat Topal, Anil Bas, and Imke van Heerden. 2021. Exploring transformers in natural language generation: Gpt, bert, and xlnet. *arXiv preprint arXiv:2102.08036* (2021).

[62] Minh SQ Truong, Eric Chen, Deanyone Su, Liting Shen, Alexander Glass, L Richard Carley, James A Bain, and Saugata Ghose. 2021. RACER: Bit-pipelined processing using resistive memory. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 100–116.

[63] Weier Wan, Rajkumar Kubendran, S Burc Eryilmaz, Wenqiang Zhang, Yan Liao, Dabin Wu, Stephen Deiss, Bin Gao, Priyanka Raina, Siddharth Joshi, et al. 2020. 33.1 A 74 TMACS/W CMOS-RRAM neurosynaptic core with dynamically reconfigurable dataflow and in-situ transposable weights for probabilistic graphical models. In *2020 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 498–500.

[64] Alex Wang. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461* (2018).

[65] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. arXiv:1804.07461 [cs.CL]

[66] Hanrui Wang, Zhanghao Wu, Zhijian Liu, Han Cai, Ligeng Zhu, Chuang Gan, and Song Han. 2020. Hat: Hardware-aware transformers for efficient natural language processing. *arXiv preprint arXiv:2005.14187* (2020).

[67] Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2024. Augmenting language models with long-term memory. *Advances in Neural Information Processing Systems* 36 (2024).

[68] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. 2024. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 330–344.

[69] H-S Philip Wong, Simone Raoux, SangBum Kim, Jiale Liang, John P Reifenberg, Bipin Rajendran, Mehdi Asheghi, and Kenneth E Goodson. 2010. Phase change memory. *Proc. IEEE* 98, 12 (2010), 2201–2227.

[70] Yuting Wu, Ziyu Wang, and Wei D Lu. 2024. PIM GPT a hybrid process in memory accelerator for autoregressive transformers. *npj Unconventional Computing* 1, 1 (2024), 4.

[71] Cheng-Xin Xue, Je-Min Hung, Hui-Yao Kao, Yen-Hsiang Huang, Sheng-Po Huang, Fu-Chun Chang, Peng Chen, Ta-Wei Liu, Chuan-Jia Jhang, Chin-I Su, et al. 2021. 16.1 A 22nm 4Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7 TOPS/W for tiny AI edge devices. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. IEEE, 245–247.

[72] Guowei Yang, Cansu Demirkiran, Zeynep Ece Kizilates, Carlos A Ríos Ocampo, Ayse K Coskun, and Ajay Joshi. 2023. Processing-in-memory using optically-addressed phase change memory. In *2023 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–6.

[73] Haichao Yang, Chang Eun Song, Weihong Xu, Behnam Khaleghi, Uday Mallappa, Monil Shah, Keming Fan, Mingu Kang, and Tajana Rosing. 2024. FSL-HDnn: A 5.7 TOPS/W End-to-end Few-shot Learning Classifier Accelerator with Feature Extraction and Hyperdimensional Computing. In *2024 IEEE European Solid-State Electronics Research Conference (ESSERC)*. IEEE, 33–36.

[74] Xiaoxuan Yang, Bonan Yan, Hai Li, and Yiran Chen. 2020. ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.
[75] Xiaoxuan Yang, Bonan Yan, Hai Li, and Yiran Chen. 2020. ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration. In *Proceedings of the 39th International Conference on Computer-Aided Design* (Virtual Event, USA) *(ICCAD '20)*. Association for Computing Machinery, New York, NY, USA, Article 92, 9 pages. doi:10.1145/3400302.3415640
[76] Peng Yao, Huaqiang Wu, Bin Gao, Jianshi Tang, Qingtian Zhang, Wenqiang Zhang, J Joshua Yang, and He Qian. 2020. Fully hardware-implemented memristor convolutional neural network. *Nature* 577, 7792 (2020), 641–646.
[77] Amir Yazdanbakhsh, Ashkan Moradifirouzabadi, Zheng Li, and Mingu Kang. 2022. Sparse attention acceleration with synergistic in-memory pruning and on-chip recomputation. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 744–762.
[78] Kentaro Yoshioka. 2024. vision-transformers-cifar10: Training Vision Transformers (ViT) and related models on CIFAR-10. https://github.com/kentaroy47/vision-transformers-cifar10.
[79] Shimeng Yu, Wonbo Shim, Xiaochen Peng, and Yandong Luo. 2021. RRAM for compute-in-memory: From inference to training. *IEEE Transactions on Circuits and Systems I: Regular Papers* 68, 7 (2021), 2753–2765.
[80] Wenqiang Zhang, Bin Gao, Jianshi Tang, Peng Yao, Shimeng Yu, Meng-Fan Chang, Hoi-Jun Yoo, He Qian, and Huaqiang Wu. 2020. Neuro-inspired computing chips. *Nature electronics* 3, 7 (2020), 371–382.
[81] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. Transpim: A memory-based acceleration via software-hardware co-design for transformer. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1071–1085.
[82] Mohammed A Zidan, John Paul Strachan, and Wei D Lu. 2018. The future of electronics based on memristive systems. *Nature electronics* 1, 1 (2018), 22–29.

Appendix

A. Abstract

This section describes the process to reproduce the key experimental results of the BERT-Base, BERT-Large, and Llama3 models shown in Figure 12, focusing on inference performance degradation under analog noise in a hybrid RRAM-based processing-in-memory (PIM) accelerator. Our simulator emulates the analog non-idealities by injecting noise into the weight matrices and measures the inference loss to evaluate the effectiveness of single-level cell (SLC) mapping based on gradient importance. The full workflow includes: (1) model and dataset preparation, (2) pre-SVD fine-tuning (optional), (3) low-rank decomposition using singular value decomposition (SVD) for linear layers and post-SVD truncation, (4) fine-tuning with gradient redistribution, and (5) noise-injected inference to evaluate the accuracy or loss.

B. Artifact Check-list

- **Program**: HyFlexPIM noise-aware simulator (this repository).
- **Compilation**: Not required (Python-based Jupyter Notebook).
- **Model & Dataset**: meta-llama/Llama-3.2-1B (Llama3) with PTB dataset (default) and bert-large-uncased (bert-base-uncased) with GLUE dataset, user-customizable.
- **Run-time environment**: Python 3.8.2+, Jupyter Notebook. Linux (e.g., Ubuntu 20.04+) is recommended for GPU training.
- **Hardware**: Used NVIDIA RTX A6000 GPU with 48GB VRAM (used for fine-tuning).
- **Metrics**: Inference accuracy or loss under different device noise levels and the proportion of weights mapped to SLC.
- **Output**: Accuracy or loss & perplexity.
- **Disk space required**: Approximately 30 GB (for model checkpoints and gradients).
- **Total experiment time**: Approximately 2–4 hours (depending on fine-tuning).
- **Publicly available**: Yes.
- **License**: MIT License.
- **DOI**: `10.5281/zenodo.15103949`.

C. Description

1) How to access:
All source code, notebook, and utility files are available at:
https://github.com/songchangeun96/HyFlexPIM
2) Hardware dependencies:
We recommend using a GPU with at least 48 GB of memory for training steps. Our experiments were conducted using two NVIDIA RTX A6000 GPUs (48 GB each). No custom hardware is required.
3) Software dependencies:
Python 3.8.2+, with packages listed in `requirements.txt`. The simulation runs via Jupyter Notebook and uses PyTorch as its core ML framework.

D. Installation

```
$ git clone
  https://github.com/songchangeun96/HyFlexPIM.git
$ cd HyFlexPIM
$ pip install -r requirements.txt
$ jupyter notebook HyFlexPIM_llama3.ipynb
 (or jupyter notebook HyFlexPIM_BERT.ipynb)
```

E. Experiment Workflow

The following steps reproduce the result of Llama3 in Figure 12 (Same steps are applied to the BERT model):

Step 1: Load the default LLaMA-3.2 1B model and PTB dataset (BERT-Large or BERT-Base model and GLUE dataset for BERT implementation). Optionally, evaluate with `evaluate_model()`. Users may also substitute the default model with a quantized custom model of their choice.
Step 2: Fine-tune for 2–3 epochs to improve noise robustness, then merge weights using `model.merge_and_unload()` to apply all fine-tuned parameters to the base model before noise injection.
Step 3: Apply SVD to linear layers and truncate by using `replace_linear_layer_llama()`.
Step 4: Fine-tune the model and save gradient of Σ to `./gradient/`.
Step 5: Reload gradients and inject noise using `apply_noise_to_llama(model, std, th)`, and evaluate loss.

F. Evaluation and Expected Results

This simulator measures inference loss (accuracy) under analog noise perturbations for the Llama3 (BERT) models. Initially, applying SVD followed by truncation introduces significant loss (accuracy degradation) due to the reduced rank approximation. Subsequent fine-tuning for a few epochs helps recover performance, resulting in only a modest (approximately 0.2–0.3) increase in loss

(less than 1% of accuracy degradation for BERT) compared to the original pre-truncation model.

After fine-tuning, our noise-injection simulator allows users to evaluate model robustness under varying levels of analog memory noise and SLC configurations. By default, it simulates RRAM-based memory behavior with a noise standard deviation of `std` = 0.025. Additionally, users can adjust the threshold parameter `th` to specify the desired proportion of weights stored in SLC. Analog noise is then selectively injected only into the remaining weights mapped to low-precision MLC. While the simulator is currently tailored to the BERT summarization and Llama3 generative model, we plan to extend the support of the `replace_linear_layer` function to a broader range of models by improving compatibility in future updates.